RAPPORT FINAL GLOBAL

Architecture, Développement et Déploiement d'Applications Distribuées Big Data

Intitulé du module : Applications Distribuées

Parcours : Big Data

Nom Complet : BAZARE DAVID

Année Universitaire : 2025 - 2026

Introduction

Le développement contemporain des architectures logicielles et l'avènement du Big Data ont radicalement transformé la gestion des systèmes d'information. Aujourd'hui, l'enjeu des entreprises n'est plus uniquement de stocker la donnée de manière passive, mais bien d'être en mesure de l'ingérer, de la traiter, de la distribuer et de l'analyser en temps réel avec des garanties strictes de performance et de tolérance aux pannes.

Dans le cadre du parcours Big Data, le module Applications Distribuées a permis d'appréhender ces concepts complexes non pas de façon théorique, mais à travers une démarche d'ingénierie logicielle itérative et rigoureuse. L'objectif global de ce semestre a été de concevoir de bout en bout une infrastructure robuste de traitement de flux de données événementielles, capable de monter en charge et de garantir la continuité de service en cas de panne.

Ce document constitue le rapport final global synthétisant l'intégralité des huit Travaux Pratiques réalisés. Chaque étape a représenté une brique essentielle du pipeline de données final, allant de la validation stricte des contrats de messages à l'orchestration finale par graphes, illustrant une maîtrise complète du cycle de vie de la donnée distribuée.

Synthèse des Travaux Pratiques (TPs)

TP 1 — Modélisation et Contrats de Messages

Contexte	Définition de la structure initiale des messages circulant au sein du système pour éviter la corruption de données.
Objectifs	Formaliser des contrats de données stricts pour les entités clés (Commandes, Paiements, Snapshots de performance).
Outils Utilisés	Python 3, Dataclasses, Typage statique, Modules de sérialisation JSON.
Démarche Suivie	Création de structures de données immuables à l'aide de dataclasses Python. Implémentation de mécanismes de validation de types, de présence de champs obligatoires et de contraintes métiers (ex: montants strictement positifs, formats d'identifiants).
Résultats Obtenus	Garantie qu'aucune donnée malformée ne puisse transiter en aval. Les contrats agissent comme une barrière de sécurité et une documentation technique vivante du système.

TP 2 — Service Ingestion Synchrone et Validation

Contexte	Mise en place de la passerelle d'entrée de l'architecture distribuée recevant les flux externes.
Objectifs	Développer un service HTTP/TCP synchrone capable de valider les payloads en moins de 200ms pour libérer immédiatement le client.
Outils Utilisés	Sockets Python, Frameworks légers d'API, Client HTTP de test, multi-threading.
Démarche Suivie	Conception de routes d'ingestion dédiées. À la réception d'un payload, le service instancie le validateur de contrat. Si le contrat est respecté, un identifiant unique temporisé est attribué et une réponse '201 Created' est retournée. En cas d'erreur, le service renvoie un code '400' documentant les anomalies de validation.
Résultats Obtenus	Une passerelle d'ingestion étanche, capable de filtrer le bruit et d'aiguiller proprement les flux valides vers la file d'attente asynchrone.

TP 3 — Conception d'un Message Broker Partitionné

Contexte	Découplage des architectures point à point par la création d'un bus événementiel intermédiaire.
Objectifs	Implémenter un gestionnaire de messages (Broker) simulant le comportement de clusters comme Apache Kafka, organisé en partitions logiques indépendantes.
Outils Utilisés	Structures de données concurrentes, Modules Threading et Queue de Python.
Démarche Suivie	Développement du Broker capable de gérer des structures nommées (Topics) subdivisées en partitions. Implémentation des mécanismes d'écriture (Append-Only Log) et d'affectation de threads consommateurs par partition pour paralléliser le travail.
Résultats Obtenus	Suppression des goulots d'étranglement synchrones. Le producteur publie son message dans le Broker et reprend instantanément son exécution.

TP 4 — Routage Déterministe et Stratégies de Partitionnement

Contexte	Distribution intelligente et uniforme de la charge de données à travers les nœuds ou partitions du broker.

Objectifs	Garantir le maintien de l'ordre chronologique des événements pour une même entité métier (ex: un restaurant ou un client) tout en répartissant la charge globale.
Outils Utilisés	Algorithmes de hachage (MD5, SHA-256 via hashlib), calculs modulo.
Démarche Suivie	Développement d'une fonction de partitionnement déterministe au niveau du Producteur. La clé métier (ex: restaurant_id) est convertie en empreinte numérique via un algorithme de hachage, puis orientée vers une partition spécifique grâce à un calcul modulo le nombre de partitions actives.
Résultats Obtenus	Distribution uniforme des messages évitant l'asymétrie de charge (Data Skew), tout en assurant que toutes les étapes logistiques d'une même commande atterrissent sur la même partition dans l'ordre exact de leur émission.

TP 5 — Gestion de la Persistance des Offsets et Tolérance aux Pannes

Contexte	Sécurisation du processus de consommation de données face aux arrêts brutaux des serveurs.
Objectifs	Implémenter un cycle de commit d'offsets pour assurer une sémantique de livraison 'At-least-once' et permettre une reprise de lecture transparente après un crash.
Outils Utilisés	Fichiers de configuration JSON persistants, Gestion des accès disques atomiques, Signaux d'interruption.
Démarche Suivie	Mise en place d'un pointeur logique (Offset) pour chaque consommateur sur sa partition. Le consommateur lit le message, exécute le traitement métier, persiste son résultat, puis sauvegarde son nouvel offset sur disque. En cas de redémarrage, le consommateur charge le dernier offset validé pour reprendre le flux exact.
Résultats Obtenus	Résilience complète de la couche de consommation. Aucun événement n'est perdu ou sauté lors d'une coupure de courant ou d'une défaillance logicielle.

TP 6 — Moteur de Traitement Stream et State Stores

Contexte	Extraction de valeur métier immédiate à partir du flux continu d'événements.
Objectifs	Calculer des indicateurs de performance en temps quasi réel (moyennes glissantes, compteurs d'activité) via le maintien d'états locaux.

Outils Utilisés	Mémoire partagée, structures de fenêtrage temporel (Windowing), State Stores persistants.
Démarche Suivie	Développement d'un processeur de streaming lisant en continu les topics. Implémentation d'un magasin d'états (State Store) local pour mémoriser les valeurs intermédiaires. Mise en place d'une politique de gestion des événements tardifs basés sur un seuil de tolérance (Watermark).
Résultats Obtenus	Disponibilité immédiate des indicateurs clés (nombre de commandes en cours, temps de préparation moyen des 15 dernières minutes), mis à jour au fil de l'eau à chaque nouvel événement.

TP 7 — Stockage Multi-Couches (Medallion) et Pipelines Batch

Contexte	Persistance historique structurée de l'intégralité de l'activité à des fins d'audit et d'analyse profonde.
Objectifs	Organiser un lac de données moderne (Architecture Medallion) et concevoir des traitements batch nocturnes idempotents.
Outils Utilisés	Format JSON Lines (.jsonl), Arborescence Hive Partitioning, Scripts de calcul batch.
Démarche Suivie	Déploiement de trois zones de stockage. Bronze : stockage des payloads bruts immuables. Silver : exécution d'un batch de nettoyage, filtrage des anomalies et enrichissement géographique. Gold : agrégations décisionnelles (chiffre d'affaires par jour et par ville). L'idempotence est garantie par une stratégie d'écrasement ciblé de partition à la relance.
Résultats Obtenus	Structure de stockage hautement optimisée permettant l'élagage de partitions (Partition Pruning) lors des requêtes analytiques, réduisant les temps de calcul historiques de manière drastique.

TP 8 — Orchestration par DAG et Observabilité

Contexte	Pilotage et supervision centralisés de l'ensemble de l'infrastructure de production.
Objectifs	Orchestrer l'enchaînement logique des pipelines batch sous forme de graphe acyclique direct (DAG) et monitorer la santé du système (Lag, Latence).

Outils Utilisés	Moteurs d'orchestration de tâches, Modules de logs structurés JSON, Indicateurs de Lag.
Démarche Suivie	Modélisation du pipeline sous forme de DAG définissant les dépendances amont/aval (Vérification -> Nettoyage -> Agrégation -> Notification). Intégration de sondes d'observabilité mesurant le débit d'ingestion par seconde et le retard de consommation (Lag). Connexion à un dashboard ASCII unifié.
Résultats Obtenus	Gouvernance complète du système. Les équipes d'ingénierie disposent d'une visibilité totale sur les goulots d'étranglement et sont immédiatement notifiées en cas d'échec d'une étape.

Analyse Personnelle

Difficultés Rencontrées et Solutions Adoptées

1. Concurrence et Accès Mémoire (TP 3 & TP 6) : Lors de la parallélisation de la consommation des messages via plusieurs threads, des conflits d'accès (Race Conditions) sont apparus sur le dictionnaire global contenant les messages du Broker. Pour y remédier, j'ai implémenté des mécanismes de verrouillage chirurgical (threading.Lock). Cette approche m'a appris à équilibrer la sécurité des threads avec les performances globales du système.

2. Résolution du Désordre des Messages (TP 6) : En mode distribué, certains événements de paiement arrivaient avant la création de la commande associée en raison des latences réseau. J'ai résolu ce problème en concevant une zone tampon temporaire au sein du State Store. Les messages en avance y sont mis en attente pendant une fenêtre temporelle configurable, assurant une réconciliation chronologique parfaite.

3. Risque de Doublons lors des Reprises sur Panne (TP 5 & TP 7) : Relancer un pipeline après un crash provoquait parfois l'écriture double des indicateurs dans la zone Gold. J'ai adopté le principe d'idempotence : chaque script batch nettoie et écrase sa partition cible avant de réécrire les résultats, garantissant que le résultat final reste identique quel que soit le nombre d'exécutions.

Apprentissages Acquis

Ce semestre a consolidé mes compétences pratiques en ingénierie des données à grande échelle. Au-delà de l'écriture de code, j'ai assimilé les principes fondamentaux de la décentralisation logicielle : la conception rigoureuse des contrats de messages, l'importance cruciale du choix des clés de partitionnement, et la gestion rigoureuse des pointeurs de lecture (offsets).

Le fait de reconstruire les composants fondamentaux de frameworks industriels comme Apache Kafka (le broker), Apache Airflow (le DAG) ou Spark (le streaming) m'a permis de comprendre précisément ce qui

se passe sous le capot de ces technologies, me donnant une excellente capacité à diagnostiquer les pannes de performance et d'infrastructure.

Conclusion Générale

Le module Applications Distribuées a constitué un pilier fondamental de ma formation au sein du parcours Big Data. À travers la réalisation de ces huit travaux pratiques, l'approche est passée d'une vision modulaire isolée à la mise en production d'une véritable plateforme de données événementielle unifiée, moderne et résiliente.

Le bilan de ce semestre est extrêmement positif. J'ai pris conscience que la robustesse d'une architecture distribuée ne repose pas sur l'absence totale de défaillances — qui sont inévitables à grande échelle — mais sur sa capacité intrinsèque à tolérer les pannes, à s'auto-guérir et à garantir l'intégrité comptable et fonctionnelle des données.

Ce module m'a apporté non seulement l'autonomie technique nécessaire pour concevoir des architectures événementielles complexes, mais également une rigueur conceptuelle indispensable pour appréhender les défis futurs liés au traitement des flux de données de masse en entreprise.